

Материалы к экзамену ССФ — полностью



Все семь уроков одной страницей. Для чтения подряд и для сохранения в PDF: печать из браузера даёт готовый файл.

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Accept all

Reject non-essential

Manage preferences



1. Из чего собран Cozystack

Три слоя, четыре варианта установки и почему платформа — это просто ещё один набор объектов Kubernetes.

Начнём с вопроса, который на экзамене задают чаще любого другого: **что такое Cozystack по своей природе?** Не «для чего он», а именно из чего сделан.

Ответ короткий: это Kubernetes, которому добавили новые типы объектов. Не форк, не надстройка над чужим API, не отдельный сервер управления. Когда вы просите платформу завести базу данных, вы создаёте объект — такой же, как под или сервис, только называется он Postgres. Дальше за ним следит программа-оператор и делает всё остальное.

Это стоит уложить в голове до всего прочего, потому что из этого следует почти всё остальное. Работает `kubectl`? Работает. Работает Terraform? Работает. Права раздаются обычным RBAC? Да, обычным.

Три слоя

Платформа собрана из трёх этажей, и путать их порядок — самая частая ошибка на экзамене.

Cozystack

тенанты, базы, виртуалки, S3 — как объекты Kubernetes

Kubernetes

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Talos Linux — операционная система узлов (node OS). Её особенность в том, что у неё нет командной строки: ни SSH, ни консоли, ни пакетного менеджера. Настраивается она декларативно, через свой API: вы отправляете описание желаемого состояния, узел приводит себя к нему. Корневая файловая система при этом только для чтения.

Для администратора, привыкшего зайти на сервер и что-нибудь поправить, это поначалу неудобно. Смысл в другом: если зайти и поправить нельзя, то и состояние узла не расползается со временем. Два узла, собранные по одному описанию, останутся одинаковыми через год.

Kubernetes — управляющий кластер, который работает поверх Talos. Обычный, не переделанный: тот же API, те же контроллеры.

Cozystack — набор компонентов, устанавливаемых в этот кластер. Именно они добавляют новые типы объектов и следят за ними.

Установка в три этапа

Экзамен спрашивает не команды, а порядок и смысл этапов.

1. **Talos на узлах.** Машины загружаются с образа платформы и получают машинное описание.
2. **Kubernetes.** Из этих узлов собирается управляющий кластер.
3. **Cozystack.** В кластер ставится сама платформа, дальше она разворачивает свои компоненты сама.

Третий этап устроен любопытно: платформа не устанавливает компоненты по списку, а описывает желаемый состав и передаёт его **FluxCD**. Дальше Flux сам приводит кластер к описанию и продолжает следить, чтобы состав не расходился. Обновление платформы — это смена версии в описании, а не пересборка руками.

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Четыре варианта установки



Платформу ставят не целиком, а одним из четырёх наборов. Названия надо знать **дословно** — и вот почему это не придирка: в версии 1.5 их переименовали, а старые имена (`paas-full`, `distro-full` и подобные) остались жить в чужих статьях. Перепутать легко.

Вариант	Что это	Когда ваш случай
<code>isp-full</code>	вся платформа на Talos	своё железо, нужно всё сразу
<code>isp-full-generic</code>	то же, но на чужом Kubernetes	кластер уже есть: k3s, kubeadm, RKE2
<code>isp-hosted</code>	платформенные сервисы без сети, хранилища и виртуализации	поверх управляемого Kubernetes, где сеть и диски дал провайдер
<code>default</code>	только источники пакетов, ничего не включено	собираете платформу сами, компонент за компонентом

Запомнить проще через одну фразу: `isp-full` — всё на Talos, `-generic` — то же на других дистрибутивах, `-hosted` — только платформа, инфраструктура снизу чужая, `default` — с нуля и вручную.

Выбирают один раз, при установке. Сменить набор на работающей платформе — это не переключатель, а переустановка.

Из чего она состоит

Компонентов много, но на экзамене спрашивают **кто за что отвечает**, а не полный

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Задача	Компонент
Виртуальные машины	KubeVirt
Дисковое хранилище	LINSTOR / DRBD
Объектное хранилище (S3)	SeaweedFS
Сеть подов	Cilium
Сеть тенантов, VPC	Kube-OVN
Внешние адреса	MetalLB
Метрики	VictoriaMetrics
Журналы	VictoriaLogs
Графики	Grafana
Доставка конфигурации	FluxCD
Вход по учётным записям	Keycloak

Обратите внимание, чего в списке **нет**: Prometheus и Loki. Их часто подставляют в варианты ответа как ловушку — метрики и журналы здесь собирают VictoriaMetrics и VictoriaLogs.

Что нужно от железа

Точные цифры на экзамене не спрашивают, но порядок величин знать надо.

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Платформа собрана. Дальше её надо между кем-то поделить — этим и займётся следующий урок.



Что спросят на экзамене

- Порядок слоёв снизу вверх: Talos → Kubernetes → Cozystack.
- Что Talos управляется через API и не имеет SSH, а корень доступен только на чтение.
- Что платформа расширяет Kubernetes своими типами объектов, а не заменяет его.
- Названия четырёх вариантов установки (bundles) — дословно.
- Кто за что отвечает: KubeVirt, LINSTOR, SeaweedFS, Cilium, Kube-OVN, MetalLB.
- Что метрики хранит VictoriaMetrics, а не Prometheus.
- Что готовность установки проверяют по состоянию релизов FluxCD.

Подробнее: [обзор платформы](#) · [установка](#) · [первые шаги](#)

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.



2. Тенанты и доступ

Как платформа делит себя между тенантами, откуда берутся квоты и почему тенант — это не просто namespace.

Вторая по весу тема экзамена — и, пожалуй, самая полезная на практике. Если первый урок объяснял, из чего платформа собрана, то этот — как она делится между людьми.

Тенант — это объект, а не папка

В привычном Kubernetes изоляция начинается и заканчивается пространством имён. Здесь иначе: **тенант (tenant) — это объект платформы**, и создание его влечёт за собой целый набор последствий.

Заводите вы его так же, как всё остальное:

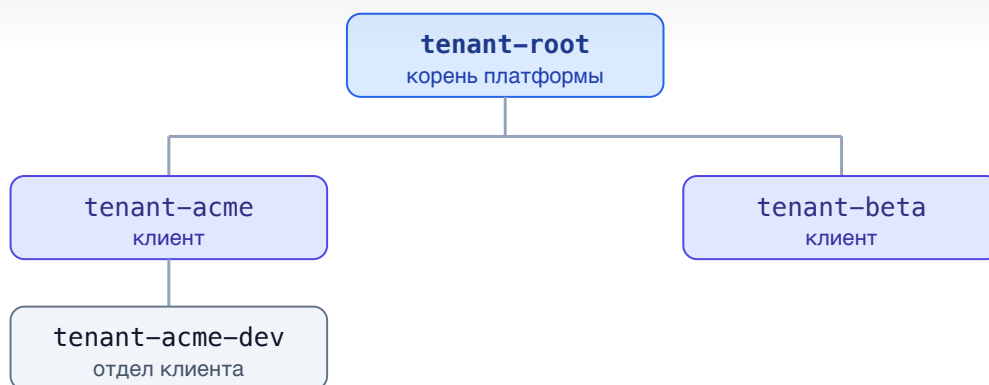
```
apiVersion: apps.cozystack.io/v1alpha1
kind: Tenant
metadata:
  name: acme
  namespace: tenant-root
```

Платформа в ответ создаёт пространство имён (namespace), раздаёт права, вешает сетевые политики, заводит квоты и, если попросили, поднимает внутри собственный мониторинг и хранилище.

Тенанты вкладываются друг в друга

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.



Тенант живёт внутри родителя. Имя пространства имён складывается из слова `tenant-` и имени.

Правило именования спрашивают, и оно чуть хитрее, чем кажется: имя пространства имён собирается из слова `tenant-` и **всей цепочки предков через дефис, кроме корня**.

Путь тенанта	Пространство имён
--------------	-------------------

root/acme	tenant-acme
root/alpha/beta	tenant-alpha-beta
root/a/b/c	tenant-a-b-c

Корень в имя не попадает — `tenant-root-alpha-beta` будет неверным ответом.

Отсюда же ограничение на имена: **дефис в имени тенанта запрещён**, потому что он занят под разделитель предков. Тенант `acme-dev` не существует — существует `dev` внутри `acme`, и живёт он в `tenant-acme-dev`.

Зачем это нужно: провайдер отдаёт клиенту `tenant`, а тот делит его между своими

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Ключевая мысль, которую любят проверять: **если сервис выключен, тенант поднимается вверх по дереву до ближайшего предка, у которого он включён**. Не обязательно до родителя и не обязательно до корня — до ближайшего. Не остаётся без него, а наследует. Тенант без своего мониторинга шлёт метрики в мониторинг родителя; тенант без своего хранилища кладёт файлы в родительское.

Включать своё имеет смысл, когда нужна настоящая изоляция — например, клиент не должен видеть чужие метрики даже теоретически. Плата за это — ресурсы: каждый включённый сервис это реально работающие процессы.

Квоты и переподписка

Тенанту назначают предел по процессорам и памяти:

```
spec:
  resourceQuotas:
    cpu: 16
    memory: 32Gi
```

А теперь то, на чём спотыкаются практически все, — **переподписка** (overcommit, `cpuAllocationRatio`, по умолчанию **10**).

Работает она так: когда вы запускаете нагрузку с пределом в четыре процессора, платформа просит у планировщика не четыре, а **предел, делённый на коэффициент** — то есть четыре десятых. Гарантируется малое, а взять при необходимости можно много.

Формула короткая, и её стоит запомнить дословно:

$$\text{запрос} = \text{предел} \div \text{cpuAllocationRatio}$$

Смысл в том, что приложения почти никогда не потребляют свой предел. Раздавать гарантии по верхней планке значит держать половину железа

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

По умолчанию tenants друг друга не видят: сетевые политики запрещают ходить между пространствами имён. И это не переключатель — **отключить изоляцию нельзя**. Флаг `isolated` существовал до версии 1.0 и убран; если он попадётся в вариантах ответа, это ловушка.

Отрезаны поды не только от соседей. По умолчанию они не достучатся ни до `kube-apiserver`, ни до собственного `etcd` тенанта. Нужен доступ — его открывают точно, метками на поде:

```
policy.cozystack.io/allow-to-apiserver: "true"  
policy.cozystack.io/allow-to-etcd: "true"
```

Значение — строка `"true"`, и это тоже спрашивают.

Как люди попадают внутрь

Три пути, и это тоже спрашивают.

Дашборд — веб-интерфейс платформы. Вход через **Keycloak**, то есть по корпоративной учётной записи, а не по отдельному паролю.

kubeconfig — файл доступа, который выдаётся тенанту. Внутри него не сертификат, а вызов внешней программы: `kubectl` при первом обращении открывает браузер, вы входите через Keycloak, и выданный токен действует до истечения срока. Отсюда, кстати, требование поставить `kubelogin`: без него `kubectl` не знает, как логиниться.

Terraform — тот же API, только описанием.

Права внутри тенанта раздаются обычными ролями Kubernetes. Никакой отдельной системы разрешений у платформы нет, и это ответ на популярный

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Что спросят на экзамене

- Что тенант — объект платформы, а не просто пространство имён.
- Что имя пространства имён = `tenant`– плюс цепочка предков через дефис, без корня.
- Что дефис в имени тенанта запрещён.
- Что тенанты вкладываются друг в друга, а вложенный создаётся внутри родителя.
- Что выключенный сервис наследуется от ближайшего предка, у которого он включён, — не обязательно от родителя.
- Что запрос нагрузки = её предел, делённый на `cpuAllocationRatio` (по умолчанию 10).
- Что изоляцию отключить нельзя, а флаг `isolated` убран в версии 1.0.
- Имена меток дословно: `policy.cozystack.io/allow-to-apiserver` и `allow-to-etcd`, значение — строка `"true"`.
- Что вход идёт через Keycloak, а права — обычным RBAC.

Подробнее: [тенанты](#) · [эксплуатация платформы](#)

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.



3. Каталог управляемых сервисов

Что можно заказать у платформы, что происходит после нажатия кнопки и почему это всегда один и тот же механизм.

Каталог — то, ради чего платформу обычно и берут. Вместо «поставьте нам PostgreSQL» человек открывает список и заказывает базу, как заказывают виртуалку.

Что в списке

Категория	Позиции
Базы данных	PostgreSQL, MySQL/MariaDB, MongoDB, Redis, ClickHouse, FoundationDB
Очереди и поиск	Kafka, NATS, RabbitMQ, OpenSearch
Хранилища	бакеты S3, диски виртуальных машин
Секреты	OpenBao
Реестр образов	Harbor
Вычисления	кластеры Kubernetes, виртуальные машины

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Вот что стоит понять один раз и больше не возвращаться.



Один и тот же путь для базы, очереди, виртуалки и кластера.

Вы создаёте объект. Платформа заводит на него HelmRelease, Flux разворачивает чарт — а уже из чарта приезжает **оператор**, который дальше и ведёт вашу базу: следит за репликацией, снимает копии, переключает главную копию при отказе.

Порядок звеньев стоит запомнить именно так: оператор приезжает **из** чарта, а не предшествует ему.

Отсюда два практических вывода, которые попадают в вопросы. Первый: **дашборд не делает ничего особенного** — нажатие кнопки собирает ровно такой же объект и отправляет его в API. Второй: если сервис не поднялся, смотреть надо на HelmRelease, а не гадать по подам.

Три пути к одному и тому же

Дашборд — форма с полями. Поля берутся из описания приложения, поэтому список параметров всегда соответствует версии платформы.

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Что задаётся при заказе



Набор полей у каждого сервиса свой, но три вещи есть почти всегда.

Размер (`resourcesPreset`) — сколько ресурсов дать. Задаётся не числами, а готовым набором вроде `t1.micro` или `u1.medium`: платформа подставляет за ним конкретные значения.

Хранилище (`storageClass`) — сколько места и какого класса. `replicated` держит несколько копий на разных узлах, `local` быстрее, но живёт на одном.

Пользователи и базы — платформа заводит их сама. Именно поэтому в файле схемы, который вы потом накатываете, нет команд создания базы и пользователя: они уже выполнены.

Пароли платформа генерирует сама и кладёт в секрет. В дашборде он виден на вкладке `Secrets` у самого приложения.

Отказоустойчивость не бесплатна

Заказывая сервис, вы выбираете число копий. Одна копия — учебный стенд: узел ушёл, сервис ушёл с ним. Две и больше — платформа сама следит, кто главный, и переключает при отказе.

Здесь же живёт частая ловушка: **отказоустойчивость сервиса и сохранность данных — разные вещи**. Копии спасают от падения узла, но не от удалённой таблицы. Для второго нужны резервные копии, и о них отдельный урок.

Каталог мы прошли, но одну его позицию — виртуальные машины — стоит разобрать отдельно: у неё своя модель, и привычки из `vSphere` здесь подводят.

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Что спросят на экзамене

- Что заказ любого сервиса — это объект в API, а дашборд лишь собирает его за вас.
- Цепочку: объект → HelmRelease → чарт → оператор → работающие поды.
- Что диагностику начинают с состояния HelmRelease.
- Что пользователей и базы заводит платформа, а пароли кладёт в секрет.
- Разницу между replicated и local.
- Что число копий защищает от отказа узла, но не заменяет резервные копии.

Подробнее: [управляемые приложения](#) · [кластеры Kubernetes](#)

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.



4. Виртуальные машины

Как виртуалка становится обычной нагрузкой Kubernetes, зачем диск отдельно от машины и где проходят честные границы.

Тема, на которой администратору VMware проще всего — и на которой он же чаще всего попадает, потому что привычные аналогии тут работают лишь наполовину.

Машина как обычная нагрузка

Виртуализацию обеспечивает **KubeVirt**. Идея у него такая: виртуальная машина — это процесс, который запускается в поде, а управляется как всё остальное в кластере.

Из этого следует то, что для человека из мира vSphere звучит непривычно. Планировщик Kubernetes выбирает узел для машины так же, как для приложения. Ресурсы считаются теми же запросами и пределами. Права выдаются тем же RBAC.

Гипервизор при этом честный — KVM, тот же, что в любом Linux.

Два объекта вместо одного

Здесь главное отличие от привычной модели, и его спрашивают.

VMDisk

по имени

VMInstance

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Диск создаётся отдельно и живёт своей жизнью. Содержимое он берёт одним из четырёх способов, и их стоит различать: клон готового образа из каталога платформы, загрузка по ссылке HTTP, копия другого диска, выгрузка файла со своей машины.

Как попасть внутрь

Здесь заканчиваются привычки и начинается специфика.

cloud-init — стандартный механизм облачных образов: при первом запуске система читает описание и настраивает себя. Пароль, ключи, пакеты, команды. Это ближайший аналог Customization Specification, только описанный текстом рядом с самой машиной.

virtctl — отдельная утилита для консоли, экрана и проброса портов.

Важная деталь, на которой спотыкаются в реальной работе: цель называют **с префиксом типа**, а не голым именем. Префикс `vmi` — это запущенный экземпляр той самой `VMInstance` со схемы выше: объект каталога описывает машину, а `vmi` — её работающее воплощение.

```
virtctl console --namespace=tenant-acme vmi/vm-instance-app
```

Причина в правах. Тенанту выдан доступ к консоли как к подресурсу запущенного экземпляра, а не к объекту машины целиком. Голое имя попадает не туда и возвращает отказ.

Что умеет и чего не умеет

Экзамен любит вопросы про возможности — и здесь важно не перепутать, чего платформа не делает, с тем, что она делает иначе.

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Чего действительно нет — **автоматической балансировки нагрузки между узлами**. В vSphere этим занимается DRS: он сам решает, что машину пора перевезти, и везёт. Здесь такого нет. Планирование ёмкости и перевоз машин во время обслуживания делаются осознанно, а не сами собой.

Разница тонкая, но именно на ней строят вопросы: миграция как **инструмент** есть, миграция как **постоянно работающая автоматика** — нет.

Машина работает. Пакеты до неё довозит сеть — а там четыре компонента, которые легко перепутать между собой.

Что спросят на экзамене

- Что виртуализацию обеспечивает KubeVirt, а машина — обычная нагрузка кластера.
- Что диск и машина — два отдельных объекта, и диск переживает машину.
- Четыре источника содержимого диска: образ из каталога, ссылка, копия диска, загрузка файла.
- Что первичная настройка идёт через cloud-init.
- Что `virtctl` требует префикса `vmi/`, и почему — из-за прав на подресурс.
- Что живая миграция, снимки состояния и проброс видеокарт есть.
- Что автоматической балансировки между узлами, как у DRS, нет.

Подробнее: [виртуализация](#) · [хранилище](#)

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.



5. Сети

Четыре компонента, каждый со своей задачей: кто носит пакеты внутри, кто делит сети между тенантами, кто выдаёт внешние адреса и кто пускает HTTP.

Сетевая часть выглядит сложной, пока не разложить её по задачам. Компонентов четыре, и у каждого своя работа — экзамен спрашивает именно распределение ролей, а не настройки.

Ingress / Gateway

пускает HTTP снаружи внутрь, отдельно у каждого тенанта

MetalLB

выдаёт внешние адреса на своём железе, без облачного балансировщика

Kube-OVN

отдельные сети тенантов, VPC, выдача адресов

Cilium

носит пакеты между подами, применяет сетевые политики

Снизу вверх: от пакетов между подами до входа снаружи.

Cilium — основа

Отвечает за то, чтобы поды видели друг друга, и за сетевые политики. Работает на **eBPF** — технологии, которая позволяет исполнять программы прямо в ядре Linux, не переключаясь в пространство пользователя на каждый пакет.

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Разделение труда между ним и Kube-OVN стоит запомнить, потому что оба называются «сетью»: **Cilium — основной CNI**, он даёт сеть подам и применяет политики для всех. **Kube-OVN работает поверх** и нужен там, где требуются отдельные сети со своими диапазонами: сети виртуальных машин и VPC тенантов. На одном поде они не пересекаются.

Kube-OVN — сети тенантов

Cilium даёт одну плоскую сеть на кластер. Этого мало, когда тенантов много и каждому нужна своя, с собственными диапазонами адресов, — возможно, пересекающимися с чужими.

Этим занимается Kube-OVN: отдельные виртуальные сети, свои диапазоны, выдача адресов, маршрутизация между сетями там, где она разрешена.

Ближайшая аналогия из привычного мира — VPC у облачных провайдеров или сети NSX.

MetalLB — внешние адреса

В публичном облаке сервис типа LoadBalancer получает адрес от провайдера. На своём железе провайдера нет, и без MetalLB такой сервис вечно висит в ожидании.

MetalLB держит пул адресов и раздаёт их сервисам, а затем объявляет их в сеть — либо по ARP, либо через BGP, если сеть построена на маршрутизации.

Именно он нужен, чтобы виртуалка или приложение получили адрес, доступный снаружи кластера.

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

пользуется родительским, как и с остальными сервисами.



Имена и сертификаты

Два помощника, которые обычно не замечают, пока они работают.

CoreDNS отвечает за имена внутри кластера. Именно поэтому приложения обращаются друг к другу по именам вида `postgres-db-rw`, а не по адресам: имя переживает переезд базы на другой узел, а адрес — нет.

cert-manager выпускает сертификаты и продлевает их сам, без напоминаний.

Всё это работает ровно до первого сбоя. О том, как о сбое узнать и что стоит подготовить заранее, — следующий урок.

Что спросят на экзамене

- Что Cilium носит пакеты между подами и работает на eBPF.
- Что Kube-OVN даёт отдельные сети тенантов и VPC.
- Что MetalLB выдаёт внешние адреса на своём железе, объявляя их по ARP или BGP.
- Что точка входа для HTTP заводится у каждого тенанта отдельно.
- Что имена внутри кластера разрешает CoreDNS, а сертификаты выпускает cert-manager.
- Почему в настройках приложений пишут имена сервисов, а не адреса.

Подробнее: [сети платформы](#)

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.



6. Наблюдаемость и резервные копии

Кто собирает метрики и журналы, где их смотреть и чем резервная копия отличается от отказоустойчивости.

Две темы в одном уроке, потому что обе про одно: что делать, когда что-то пошло не так, — и что вы успели подготовить заранее.

Метрики и журналы

Стек здесь не тот, который ожидают по привычке, и это любимая ловушка экзамена.

Что собирают	Чем
Метрики	VictoriaMetrics
Журналы	VictoriaLogs
Графики	Grafana
Оповещения	Alerta

Ни Prometheus, ни Loki, ни Elasticsearch в платформе нет. VictoriaMetrics при этом понимает язык запросов PromQL — то есть привычные запросы работают, а

уменьшение размера комплекта и переход на более новые версии

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.



Тот же путь у журналов, только вместо vmagent — fluent-bit, а вместо VictoriaMetrics — VictoriaLogs.

Про наследование помните из второго урока: тенант без своего мониторинга шлёт метрики родительскому. А вот включать сбор **задним числом бесполезно** — записей за прошлое взять неоткуда, и это стоит решать при создании кластера, а не когда график понадобился.

Резервные копии

Здесь два разных механизма, и путать их — верный способ ошибиться в вопросе.

Копии управляемых сервисов (backup). Здесь работают **четыре объекта**, и путать их — верный способ ошибиться:

Объект

Что делает

BackupClass	куда и как складывать. Заводит администратор платформы, действует на весь кластер
-------------	---

Plan	расписание: снимать по такому-то графику
------	--

BackupJob	разовый запуск, здесь и сейчас
-----------	--------------------------------

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Начиная с версии 1.5 в платформе есть готовый `cozy-default` — класс копий, который работает сразу, без настройки хранилища.



Velero. Уровнем выше: копирует объекты самого кластера и тома. Это про восстановление платформы, а не отдельной базы.

Где проходит граница

Самое ценное в этой теме — понимать, чего резервные копии **не** делают.

Копии — не отказоустойчивость. Реплики спасают от падения узла, копии — от удалённых данных. Это разные беды, и одно другое не заменяет.

Копия, которую ни разу не разворачивали, — не копия, а надежда. Восстановление надо пробовать, пока оно не понадобилось.

И главное: копия хранится в объектном хранилище. Если оно живёт в том же кластере, что и данные, то от потери кластера целиком она не спасёт. Для настоящей защиты хранилище должно быть снаружи.

Мы разобрали, из чего платформа сделана и что она умеет. Остался последний вопрос — почему она устроена именно так.

Что спросят на экзамене

- Что метрики хранит VictoriaMetrics, а журналы — VictoriaLogs. Не Prometheus и не Loki.
- Что VictoriaMetrics совместима с PromQL.
- Что метрики собирает vmagent.
- Четыре объекта: BackupClass, Plan (расписание), BackupJob (разовый запуск), Backup (результат).

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

7. Как это вообще работает

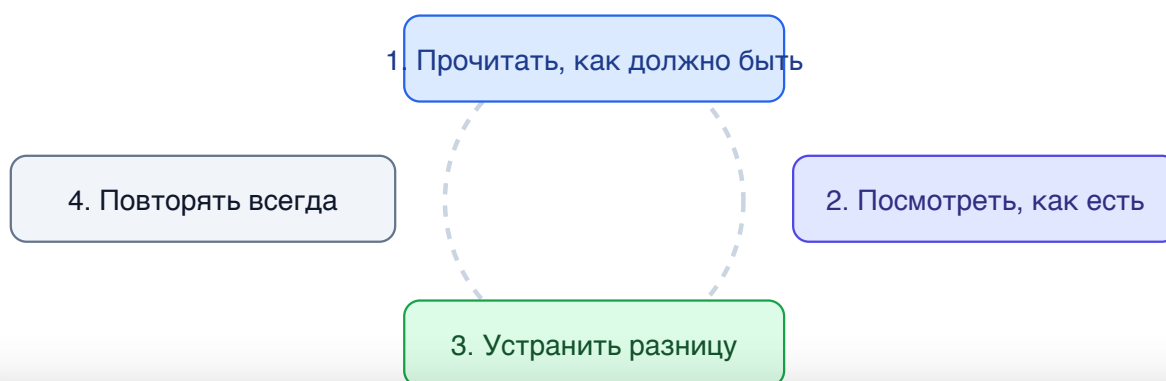
Желаемое состояние, операторы и GitOps — четыре идеи, на которых держится всё остальное.

Последний урок — про идеи, а не про компоненты. Они лежат в основании всего, о чём шла речь раньше, и вопросы по ним формулируются просто: «почему это работает именно так».

Вы описываете результат, а не действия

Привычный подход к администрированию — последовательность шагов: поставь, настрой, запусти, проверь. Здесь иначе: вы описываете, **как должно быть** — желаемое состояние, *desired state*, а система сама решает, что для этого сделать.

Разница проявляется, когда что-то ломается. Скрипт, отработавший вчера, сегодня не поможет: он уже выполнен. А описание желаемого состояния действует постоянно — если реальность от него отклонилась, её вернут обратно.



We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

`kubectl get tenants` работает, хотя в Kubernetes никаких тенантов нет. Дело в **CRD** (custom resource definition) — механизме, которым в API добавляют собственные типы.

Но тип сам по себе ничего не делает: это запись, которую кластер согласен хранить. Работу выполняет **оператор** (operator) — программа, которая следит за объектами своего типа и приводит мир в соответствие.

Отсюда практический вывод: если объект создан, а ничего не произошло, вопрос не к объекту, а к оператору. Он либо не запущен, либо не смог.

GitOps

Раз состояние описывается текстом, текст можно хранить в системе контроля версий. Тогда Git становится единственным источником истины, а специальная программа следит, чтобы кластер ему соответствовал.

Платформа использует **FluxCD** и применяет этот подход **к самой себе**: её компоненты описаны и разворачиваются тем же способом, каким вы разворачивали бы своё приложение.

Что это даёт на практике: у любого изменения есть автор и время, откат — это возврат к прошлому состоянию файла, а правка, сделанная руками в обход Git, будет отменена при следующей сверке. Последнее раздражает ровно до первого случая, когда чужая ручная правка что-нибудь роняет.

Helm

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Складывается такая цепочка, и она стоит того, чтобы её запомнить целиком:

объект → HelmRelease → чарт → оператор → работающие поды

Всё, что делает платформа, проходит по ней. Заказали базу — прошло по ней. Создали тенант — прошло по ней. Установилась сама платформа — тоже.

Что спросят на экзамене

- Что описывается желаемое состояние, а не последовательность действий.
- Что контроллер непрерывно сверяет желаемое с фактическим и устраняет разницу.
- Что CRD добавляет тип, а работу выполняет оператор.
- Что платформа применяет GitOps к самой себе через FluxCD.
- Что ручная правка в обход Git будет отменена при следующей сверке.
- Что чарт — единица упаковки, и каждая позиция каталога это чарт.
- Цепочку: объект → HelmRelease → чарт → оператор → поды.

Подробнее: [обзор платформы](#) · [концепции FluxCD](#)

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.